

BBDD Vectoriales · Memoria de la Sesión 3

Índice de contenidos

0. Introducción
 1. Qué convierte un índice en una base de datos vectorial
 2. El contrato matemático del espacio vectorial
 3. Índices ANN y búsqueda filtrada
 4. Modelo de datos e identidad
 5. Escrituras, actualizaciones y borrados
 6. Persistencia, consistencia y distribución
 7. Despliegue, seguridad y observabilidad
 8. Pinecone
 9. Chroma
 10. Weaviate
 11. Milvus
 12. Qdrant
 13. LangChain como capa de abstracción
 14. Cómo elegir una base de datos vectorial
 15. Referencias
-

0. Introducción

Una base de datos vectorial no aparece en el momento en que somos capaces de calcular un embedding. Tampoco aparece, por sí sola, cuando construimos un índice HNSW o IVF y conseguimos recuperar vecinos con una latencia razonable. Esas dos piezas son imprescindibles, pero todavía describen un problema mucho más pequeño que el que debe resolver una base de datos.

El modelo de embeddings define una representación: transforma cada objeto en un punto de un espacio de alta dimensión. La métrica establece cómo se compara una consulta con esos puntos. El índice ANN reduce el trabajo necesario para encontrar candidatos próximos. Una base de datos vectorial toma esas piezas y las integra dentro de un sistema que debe conservar estado, aceptar cambios, asociar vectores con documentos y metadatos, atender a varios clientes, aplicar filtros, recuperarse después de una caída y explicar qué está ocurriendo cuando algo no funciona.

Esta diferencia es más profunda de lo que parece. Si solo tenemos una matriz de embeddings y una biblioteca de búsqueda, la aplicación debe encargarse de relacionar cada posición de la matriz con una entidad de negocio, guardar los textos, construir filtros, decidir cómo se actualiza un registro, serializar el índice y coordinar el acceso concurrente. En una base de datos vectorial, una parte considerable de esas responsabilidades se desplaza hacia el motor. No desaparecen: cambian de propietario y adquieren una semántica explícita.

Por eso no tiene demasiado sentido reducir el estudio de estas bases a una colección de llamadas de SDK. Saber escribir `upsert()` o `query()` permite completar una demostración, pero no permite razonar sobre el sistema. Para comprender una base vectorial necesitamos entender qué representa una colección, dónde se conservan los datos, qué estructura responde a la consulta, cómo se combinan los filtros con el índice, cuándo se considera visible una escritura, qué se replica, qué se divide en shards y qué parte de todo ello administra el proveedor.

Los motores que estudiaremos ocupan posiciones deliberadamente diferentes. Pinecone representa una base gestionada en la que gran parte de la arquitectura queda detrás de un contrato de servicio. Chroma prioriza una experiencia sencilla que puede comenzar dentro del propio proceso y crecer hacia una arquitectura distribuida. Weaviate combina un modelo de objetos, un índice vectorial y estructuras invertidas para filtrado y búsqueda léxica. Milvus expone una arquitectura especialmente amplia, con separación entre almacenamiento y cómputo y una oferta extensa de índices. Qdrant organiza su funcionamiento alrededor de puntos, payloads, segmentos y un HNSW estrechamente integrado con los filtros.

No existe un ganador universal entre ellos. Cada producto decide qué debe ser sencillo, qué debe ser configurable y qué coste operativo está dispuesto a trasladar al usuario. Una característica puede ser una fortaleza para un equipo y una desventaja para otro. Ocultar el algoritmo ANN, por ejemplo, reduce el número de decisiones operativas, pero también impide aplicar un ajuste muy específico. Exponer decenas de índices ofrece control, aunque obliga a comprenderlos, medirlos y mantenerlos.

La finalidad de esta memoria es construir el marco teórico que permite interpretar esas decisiones. Los motores no se presentan como cinco tutoriales independientes, sino como cinco respuestas distintas a las mismas preguntas fundamentales: cómo se modelan los datos, cómo se localizan vecinos, cómo se filtran, cómo se materializan los cambios, cómo se escala y quién se responsabiliza de que el servicio siga disponible.

1. Qué convierte un índice en una base de datos vectorial

Un índice vectorial es una estructura de acceso. Su pregunta central es muy concreta: dado un vector de consulta, ¿qué elementos del conjunto se encuentran más próximos según una determinada función de distancia o similitud? FAISS, por ejemplo, ofrece varias respuestas algorítmicas a esa pregunta, desde la exploración exacta hasta familias IVF, HNSW o cuantizadas [1].

Una base de datos vectorial debe responder, además, a preguntas que ya no son exclusivamente geométricas. ¿Qué documento corresponde al vecino recuperado? ¿Cómo se restringe una búsqueda a una marca, un idioma o un usuario? ¿Qué sucede si dos procesos actualizan el mismo registro? ¿Puede leerse inmediatamente una escritura que acaba de ser aceptada? ¿Dónde reside el estado después de apagar el servicio? ¿Cómo se restaura una colección? ¿Qué usuario puede borrarla? ¿Cómo se reparte cuando deja de caber en una máquina?

La diferencia entre ambas categorías no consiste, por tanto, en que una base de datos utilice una búsqueda más sofisticada. De hecho, muchas bases emplean internamente los mismos algoritmos estudiados al trabajar con una biblioteca ANN. Lo que cambia es el entorno de responsabilidades que rodea al índice.

1.1. El registro vectorial

La unidad lógica de una base vectorial suele contener cuatro partes:

- Un identificador estable que permite recuperar, actualizar o eliminar la entidad.
- Uno o varios vectores sobre los que se ejecuta la búsqueda de similitud.
- Un documento, contenido o referencia que explica qué representa el vector.
- Un conjunto de metadatos estructurados que permite filtrar, ordenar, agrupar o aplicar políticas.

Cada motor utiliza su propio vocabulario. Qdrant habla de puntos y payloads. Weaviate almacena objetos con propiedades. Chroma separa IDs, embeddings, documentos y metadatos. Milvus define campos dentro de un esquema. Pinecone almacena registros dentro de namespaces. La terminología cambia, pero la necesidad es la misma: un vecino geométrico debe volver a convertirse en una entidad comprensible para la aplicación.

Esta asociación no es un detalle administrativo. Si la matriz de embeddings se reordena y el mapeo de IDs no se actualiza, el índice seguirá devolviendo distancias numéricamente coherentes, pero la aplicación mostrará documentos equivocados. Desde fuera parecerá un fallo semántico del modelo, aunque el problema real sea la integridad referencial entre el vector y sus metadatos.

1.2. Plano de datos y plano de control

Conviene separar dos planos porque exigen permisos, frecuencias y garantías distintas.

El **plano de datos** atiende el tráfico cotidiano: insertar registros, obtenerlos por ID, actualizar metadatos, ejecutar búsquedas, aplicar filtros y eliminar puntos. Estas operaciones aparecen en el camino directo de una petición de usuario y suelen estar sometidas a objetivos de latencia y throughput.

El **plano de control** administra los recursos sobre los que funciona ese tráfico: crear colecciones, fijar dimensión y métrica, configurar índices, definir shards y réplicas, gestionar credenciales, crear snapshots o eliminar un índice completo. Son operaciones menos frecuentes, pero su impacto es mucho mayor. Borrar una colección no es una variante más de borrar un registro.

Esta separación también aclara el alcance de las abstracciones. Una librería puede homogeneizar razonablemente la operación “devuélveme documentos parecidos”, pero resulta mucho más difícil construir una interfaz común para configurar HNSW, restaurar un snapshot, mover una réplica o interpretar los límites de facturación de varios proveedores. Cuanto más nos acercamos al plano de control, más importante se vuelve el SDK nativo.

1.3. El recorrido de escritura

Cuando una aplicación envía un nuevo registro, el vector no tiene por qué llegar inmediatamente a la estructura ANN definitiva. En muchos motores, la escritura se registra primero en un log duradero, se incorpora a una estructura mutable y, posteriormente, se compacta o se integra en segmentos indexados. Este diseño permite aceptar cambios sin reconstruir por completo un índice grande ante cada inserción.

La respuesta correcta a “¿ya se ha escrito?” depende entonces de qué etapa observamos. El servidor puede haber recibido la petición. El log puede haberla hecho duradera. Una lectura por ID puede encontrarla. Una búsqueda vectorial puede seguir consultando una versión anterior del índice. En un sistema distribuido, unas réplicas pueden haber aplicado el cambio y otras no.

Por esta razón, una confirmación de escritura y la visibilidad en búsqueda son propiedades relacionadas, pero no idénticas. Los productos que ofrecen consistencia eventual hacen explícita esta separación. Los que permiten elegir niveles de consistencia trasladan al cliente parte del intercambio entre actualidad, disponibilidad y latencia.

1.4. El recorrido de lectura

Una búsqueda tampoco es una única comparación. El cliente serializa la consulta y la envía a una API. El motor identifica la colección, el namespace o los shards implicados. Los filtros estructurados delimitan candidatos. El índice ANN explora una parte del espacio. Si existen varios segmentos o shards, se obtienen rankings parciales y se fusionan. Finalmente se recuperan los payloads o documentos que acompañan a los IDs seleccionados.

Cada etapa puede introducir latencia o error. El índice puede perder un vecino exacto. Un filtro mal modelado puede excluirlo. Una réplica desactualizada puede no conocer una escritura reciente. Una fusión distribuida puede requerir más candidatos locales para no perder elementos del top global. La aplicación puede interpretar una distancia como si fuera una similitud.

Entender este recorrido permite abandonar explicaciones demasiado vagas. “La base devuelve malos resultados” no es todavía un diagnóstico. Primero necesitamos saber si el espacio vectorial considera relevantes esos resultados, si el ANN reproduce ese espacio, si el filtro define el universo correcto y si la respuesta se ha interpretado respetando la semántica del score.

1.5. Base vectorial y motor de búsqueda

Las fronteras entre una base vectorial, un motor de búsqueda y una base de datos generalista son cada vez menos nítidas. Algunos motores vectoriales incorporan búsqueda BM25, índices invertidos, datos geográficos o múltiples vectores por entidad. A la vez, PostgreSQL, Elasticsearch, OpenSearch y otras bases tradicionales han añadido tipos vectoriales e índices ANN.

La etiqueta comercial no basta para elegir. Lo relevante es comprobar si el sistema ofrece el contrato requerido: calidad de recuperación, filtros, mutabilidad, persistencia, concurrencia, escalabilidad, seguridad y operación. Una extensión vectorial dentro de una base ya conocida puede ser la mejor decisión si evita duplicar datos y satisface el volumen. Una base especializada puede ser preferible cuando el vector es el patrón de acceso dominante y se necesita un control o una escala que la solución generalista no proporciona.

2. El contrato matemático del espacio vectorial

La base de datos no decide qué significa semánticamente un vector. Recibe una representación construida por un modelo y organiza la recuperación conforme a una métrica. Por ello, dimensión, preprocesamiento, normalización y función de similitud deben tratarse como parte del esquema, aunque no se parezcan a una columna convencional.

2.1. Dimensión y compatibilidad

Si un encoder produce vectores de dimensión d , cada registro contiene una secuencia de d componentes. Una colección creada para otra dimensión suele rechazar la escritura. Es un fallo saludable: el contrato se rompe de manera visible.

Más peligroso es sustituir el modelo por otro que produce la misma dimensión. La base aceptará los nuevos vectores, pero puntos generados por modelos diferentes no pertenecen necesariamente al mismo espacio. Una distancia entre ellos carece del significado aprendido durante el entrenamiento. La compatibilidad no puede validarse únicamente comprobando la forma de la matriz; también deben versionarse el modelo, la plantilla de entrada, el tokenizer, la política de truncado y la normalización.

2.2. Coseno, producto escalar y distancia euclídea

La similitud coseno compara el ángulo entre dos vectores:

$$\text{COS}(\mathbf{q}, \mathbf{x}) = \frac{\mathbf{q}^T \mathbf{x}}{\|\mathbf{q}\|_2 \|\mathbf{x}\|_2}$$

Cuando consulta y documento están normalizados a norma L2 unitaria, el denominador vale uno y el coseno coincide con el producto escalar:

$$\cos(\mathbf{q}, \mathbf{x}) = \mathbf{q}^T \mathbf{x} \quad \text{si } \|\mathbf{q}\|_2 = \|\mathbf{x}\|_2 = 1$$

La distancia euclídea cuadrática también induce el mismo orden sobre vectores unitarios, porque:

$$\|\mathbf{q} - \mathbf{x}\|_2^2 = 2 - 2 \mathbf{q}^T \mathbf{x}$$

Esto no significa que las métricas sean intercambiables en cualquier conjunto. La equivalencia depende de la normalización. Si la magnitud contiene información relevante, normalizar puede destruirla. Si el modelo fue evaluado con producto escalar sin normalización, cambiar a coseno modifica el ranking.

La métrica debe elegirse según el contrato del encoder, no por costumbre. También debe comprobarse cómo la implementa el proveedor. Algunas APIs devuelven similitud, donde un valor mayor es mejor. Otras devuelven distancia, donde el mejor resultado tiene el valor menor. Incluso bajo la etiqueta “cosine”, una API puede devolver uno menos el coseno y otra el coseno directo.

2.3. El score nativo no es una moneda común

Dos motores pueden devolver el mismo orden y puntuaciones numéricamente distintas. Pueden aplicar transformaciones, cuantización, aproximación o definiciones diferentes de distancia. Por ello, comparar un 0.82 de un proveedor con un 0.82 de otro carece de sentido sin conocer la semántica exacta.

La comparación correcta se apoya en IDs y métricas de ranking, como `recall@k` frente a una búsqueda exacta o métricas de relevancia frente a juicios humanos. El score nativo sigue siendo útil dentro del mismo contrato: permite fijar umbrales, observar cambios de distribución o detectar anomalías. Lo que no debe hacerse es convertirlo en una escala universal por el simple hecho de que ambos valores se encuentren entre cero y uno.

2.4. Coste de almacenar vectores

Antes de construir ningún índice, la memoria ocupada por los vectores densos sin compresión puede aproximarse como:

$$M_{\text{vectores}} = N d b$$

donde N es el número de vectores, d la dimensión y b el número de bytes por componente. Para `float32`, $b=4$. Un millón de vectores de 1.000 dimensiones requiere aproximadamente cuatro mil millones de bytes solo para los valores, antes de contar IDs, metadatos, réplicas, índices, logs y estructuras internas.

Esta fórmula explica por qué la dimensión importa operativamente. Duplicar d duplica el almacenamiento bruto y el ancho de banda necesario para ingerir o devolver vectores. También incrementa el trabajo de cada cálculo de distancia. En un motor autogestionado, ese crecimiento se transforma en RAM, disco, I/O y tiempo de reconstrucción. En un servicio gestionado, se transforma en almacenamiento facturable y, dependiendo del proveedor, en unidades de lectura o escritura.

2.5. Cuantización

La cuantización reduce el número de bits utilizados para representar cada componente. Pasar de `float32` a una representación de ocho bits puede reducir aproximadamente por cuatro el espacio de los valores. La cuantización binaria lleva la compresión mucho más lejos. Product Quantization divide el vector en subespacios y representa cada parte mediante centroides aprendidos.

El ahorro no es gratuito. Las distancias se calculan sobre una representación aproximada y pueden alterar el ranking. Muchos sistemas compensan ese error recuperando más candidatos con la representación comprimida y reordenando un subconjunto mediante los vectores originales. Este patrón separa una etapa barata de generación de candidatos de otra más precisa de refinamiento.

No existe una compresión universalmente segura. El efecto depende de la distribución del modelo, la dimensión, la métrica y el recall exigido. Una técnica que funciona bien con embeddings de 1.536 dimensiones puede degradar mucho otro espacio de 384. La única forma defendible de elegirla es medir memoria, latencia y calidad sobre datos representativos.

3. Índices ANN y búsqueda filtrada

Una base de datos vectorial sigue necesitando una estructura que evite comparar la consulta con todos los registros. Los conocimientos sobre ANN no quedan obsoletos al introducir un motor; pasan a formar parte de su configuración interna o del contrato que delegamos.

3.1. Búsqueda exacta

La búsqueda exacta calcula la distancia a todos los vectores válidos. Su coste crece linealmente con el tamaño del conjunto, pero ofrece el ranking exacto bajo la métrica elegida. No debe confundirse “exacto” con “relevante”: el vecino matemáticamente más próximo puede ser irrelevante para el negocio si el embedding no representa bien la intención.

El índice exacto tiene dos papeles. Para colecciones pequeñas puede ser suficientemente rápido y evitar la memoria adicional de un ANN. Para colecciones grandes funciona como oráculo de evaluación sobre una muestra: permite medir cuántos vecinos pierde una configuración aproximada.

La aproximación tiene sentido cuando ahorra recursos suficientes para justificar la pérdida potencial. Construir HNSW sobre unos pocos cientos de elementos puede costar más que recorrerlos todos. Por eso algunos motores mantienen estructuras planas para segmentos pequeños y activan el índice cuando se supera un umbral.

3.2. HNSW

HNSW organiza los vectores en un grafo de proximidad con varias capas [2]. Las capas superiores contienen menos nodos y permiten realizar saltos largos. La búsqueda desciende progresivamente hacia capas más densas hasta explorar la vecindad final.

Tres parámetros aparecen de forma recurrente:

- `M` limita aproximadamente el número de conexiones de cada nodo. Aumentarlo suele mejorar la navegabilidad y el recall, pero incrementa memoria y coste de construcción.
- `efConstruction` controla cuántos candidatos se consideran al insertar cada punto. Valores altos suelen producir un grafo de mayor calidad a cambio de una ingesta más lenta.
- `efSearch`, o simplemente `ef`, determina el ancho de la exploración durante una consulta. Aumentarlo suele mejorar recall y elevar latencia.

HNSW ofrece un excelente equilibrio para top-k pequeños y datos que caben razonablemente en memoria. Su principal coste es el grafo adicional y la complejidad de mantenerlo bajo mutaciones. Los borrados suelen representarse primero mediante marcas lógicas y recuperarse mediante compactación o reconstrucción. Una carga con actualizaciones continuas puede exigir más trabajo de mantenimiento que un catálogo casi inmutable.

3.3. IVF y familias basadas en particiones

Los índices IVF agrupan los vectores alrededor de centroides. Durante la consulta no se visitan todas las listas, sino las más prometedoras. `nlist` determina el número de particiones y `nprobe` cuántas se exploran.

Un `nprobe` bajo reduce el trabajo, pero puede ignorar la región que contiene un vecino relevante. Incrementarlo aproxima el resultado a la búsqueda exacta y aumenta la latencia. Las variantes IVF pueden almacenar vectores completos, cuantizados o acompañados por una etapa de refinamiento.

Estas familias son atractivas cuando se necesita controlar explícitamente la relación entre capacidad, memoria y recall, o cuando se recupera un top-k relativamente grande. También implican más decisiones que HNSW y pueden requerir entrenamiento del cuantizador antes de construir el índice.

3.4. Índices orientados a disco

Cuando el conjunto deja de caber en RAM, limitarse a mover un índice pensado para memoria hacia un disco puede generar accesos aleatorios costosos. Algoritmos como DiskANN diseñan el grafo y la exploración teniendo en cuenta el almacenamiento secundario [3]. Parte de la información comprimida permanece en memoria y el SSD se utiliza para recuperar vecindades o refinar candidatos.

La ventaja es ampliar capacidad sin exigir que toda la estructura resida en RAM. El precio es una dependencia mucho mayor de la latencia y las IOPS del disco. El rendimiento puede cambiar drásticamente entre un SSD NVMe local y un volumen de red. “On disk” no es una propiedad binaria: debemos preguntar qué permanece en memoria, qué se pagina, cómo funciona la caché y qué patrón de acceso genera la consulta.

3.5. Cuándo puede elegirse el algoritmo

Milvus expone una gama amplia de familias, incluidas FLAT, IVF, HNSW, SCANN, DiskANN y variantes cuantizadas [20]. Weaviate ofrece índices HNSW, flat, dynamic y HFresh en su familia actual [13]. Chroma Single Node utiliza HNSW y su arquitectura distribuida utiliza SPANN [8]. Qdrant utiliza HNSW como índice denso y concentra el ajuste en su configuración, almacenamiento y cuantización [26]. Pinecone no publica un selector equivalente para elegir el algoritmo interno en sus índices serverless.

Estas diferencias determinan dónde puede aplicarse directamente el conocimiento algorítmico. En Milvus podemos elegir una familia distinta. En Weaviate podemos escoger el tipo de índice y ajustar parámetros. En Chroma Single Node o Qdrant ajustamos HNSW dentro de los límites del motor. En Pinecone delegamos esa decisión y evaluamos el servicio mediante su contrato observable.

Delegar no elimina la necesidad de comprender ANN. Seguimos necesitando medir recall, reconocer una pérdida de vecinos y distinguirla de un error del encoder. Lo que cambia es nuestra capacidad para intervenir sobre la causa interna.

3.6. El problema de los filtros

Una búsqueda real rara vez pregunta solo por proximidad. Puede exigir `brand == "Einhell"`, `locale == "es"`, una fecha posterior a cierto umbral o permisos que incluyan al usuario actual. El filtro cambia el universo sobre el que se define el top-k.

Aplicar el filtro después de recuperar diez vecinos globales es sencillo, pero incorrecto si se necesitan diez resultados válidos. Tal vez solo dos de los diez cumplan la condición, aunque existan muchos candidatos válidos en posiciones posteriores. Aumentar arbitrariamente el número recuperado reduce el problema, pero no garantiza completitud y consume recursos.

El prefiltrado construye primero el conjunto elegible y realiza la búsqueda sobre él. Si el subconjunto es pequeño, una exploración exacta puede ser eficiente. Si es grande, necesitamos combinar la estructura escalar con el índice ANN. El reto aparece cuando el filtro rompe la conectividad del grafo: los nodos que cumplen la condición pueden quedar separados por nodos descartados.

Los motores adoptan estrategias distintas. Weaviate genera una lista de IDs mediante su índice invertido y la integra con HNSW; su estrategia ACORN añade exploración multihop y puntos de entrada para filtros restrictivos [12]. Qdrant construye índices de payload y puede añadir conexiones conscientes del filtro al grafo HNSW [26]. Milvus dispone de índices escalares y el plan de búsqueda puede variar con la selectividad. Chroma y Pinecone exponen lenguajes de filtro, pero el nivel de control sobre su ejecución física es diferente.

La lección importante es que “admite filtros” no describe el rendimiento. Debemos conocer cardinalidad, selectividad, correlación con la geometría y combinaciones habituales. Un campo booleano, una marca con miles de productos y un `user_id` casi único plantean problemas diferentes. Indexar todos los metadatos también tiene coste en almacenamiento e ingesta.

4. Modelo de datos e identidad

El esquema de una base vectorial no se limita a la dimensión. También define qué entidad representa cada vector, cómo se versiona y qué campos deben participar en filtros.

4.1. Identidad estable

El ID debe sobrevivir a una reingesta, un cambio de proveedor y una reconstrucción del índice. Utilizar la posición de una fila en una matriz es frágil: cualquier reordenación cambia su significado. Un identificador de negocio puede ser suficiente si cumple las restricciones del motor. Cuando se necesita un formato común, un UUID determinista permite obtener la misma clave a partir de la misma entidad.

La identidad estable hace posible el `upsert` idempotente. Si un lote se reenvía después de un fallo, los registros se escriben sobre las mismas claves en lugar de duplicarse. Idempotencia no significa que toda la operación distribuida ocurra exactamente una vez; significa que repetir la intención produce el mismo estado final.

También permite separar actualizaciones parciales de nuevas versiones. Si cambia el título y el embedding debe recalcularse, conservar el ID representa la evolución de la misma entidad. Si necesitamos reproducir rankings históricos, puede ser preferible crear una nueva versión y mantener ambos registros durante la migración.

4.2. Colecciones, namespaces, particiones y tenants

Estos conceptos se parecen, pero no son intercambiables.

Una **colección** suele fijar el esquema vectorial y agrupar registros que pueden consultarse juntos. Un **namespace** crea una frontera lógica dentro de un índice, como ocurre en Pinecone. Una **partición** puede dividir una colección por una clave conocida para reducir el conjunto consultado. Un **tenant** representa una frontera organizativa o de aislamiento.

La elección afecta a rendimiento, seguridad y coste. Si cada cliente vive en un namespace físicamente separado, una consulta puede tocar solo sus datos. Si todos comparten una colección y se distinguen mediante metadata, cualquier error en el filtro puede exponer información y el coste de búsqueda puede depender del conjunto completo. A la vez, separar demasiado crea miles de estructuras pequeñas y complica consultas transversales.

No debe utilizarse un filtro de aplicación como único control de acceso sin analizar sus garantías. La autorización pertenece al perímetro de seguridad, mientras que el filtrado pertenece al lenguaje de consulta. Pueden cooperar, pero no son automáticamente equivalentes.

4.3. Metadatos y payload

Los metadatos sirven para restringir la recuperación y explicar el resultado. Deben diseñarse según las consultas previstas, no como un volcado indiscriminado de todos los atributos del documento.

Los tipos importan. Guardar una fecha como cadena impide utilizar comparaciones cronológicas fiables si el formato no mantiene el orden. Convertir un número a texto cambia los operadores disponibles. Los valores nulos pueden interpretarse como ausencia de campo, `null` explícito o cadena vacía. Una aplicación que pretende cambiar de motor necesita fijar estas decisiones antes de construir adaptadores.

Los metadatos muy anchos aumentan almacenamiento y coste de escritura. Devolverlos en cada búsqueda incrementa tráfico. Indexar un campo acelera ciertos filtros, pero consume recursos y ralentiza mutaciones. La regla razonable es almacenar la información necesaria para recuperar, filtrar, autorizar o trazar el objeto, manteniendo el documento canónico en el sistema que corresponda cuando sea demasiado grande.

4.4. Uno o varios vectores

Una entidad puede tener más de una representación. Un producto puede poseer un vector de título, otro de descripción y otro de imagen. Algunos motores permiten vectores nombrados con dimensiones y métricas distintas. Otros favorecen una colección por representación o esperan que la aplicación combine rankings.

Los múltiples vectores son útiles cuando cada modalidad conserva una señal diferente. También complican el esquema, el coste de almacenamiento y la evaluación. La combinación puede realizarse mediante una suma ponderada, una fusión de rankings o una consulta multietapa. No basta con almacenar más embeddings; debemos definir qué pregunta responde cada uno y cómo se juzga su contribución.

4.5. Dense, sparse e híbrido

Los vectores densos capturan similitud aprendida. Los vectores sparse o los índices léxicos conservan coincidencias explícitas de términos. Una búsqueda híbrida intenta aprovechar ambos comportamientos.

Esta combinación es importante en catálogos con modelos, referencias o códigos exactos. Un embedding puede entender “taladro sin cable”, pero una coincidencia léxica puede ser decisiva para “XGT-18V-42”. El sistema debe fusionar escalas diferentes, por ejemplo mediante Reciprocal Rank Fusion o puntuaciones calibradas.

La capacidad híbrida no convierte automáticamente un motor en mejor opción. Añade otra estructura de índice, otra forma de consulta y nuevas métricas. Debe elegirse cuando el problema demuestra que ambas señales son necesarias.

5. Escrituras, actualizaciones y borrados

Las bases vectoriales se diferencian de un índice estático precisamente cuando los datos cambian. La mutabilidad introduce decisiones sobre batching, logs, visibilidad, compactación y reconstrucción.

5.1. `insert`, `add`, `update` y `upsert`

Una inserción estricta debería fallar o ignorarse si el ID ya existe. Una actualización debería operar sobre una entidad existente. `upsert` combina ambas intenciones: crea el registro si no está y lo modifica si ya existe.

Esa comodidad puede ocultar errores. Si una aplicación utiliza un ID equivocado, `upsert` creará una entidad nueva en lugar de avisar de que la anterior no existe. Cuando la distinción entre alta y modificación tiene valor de negocio, conviene validarla antes de llamar al motor.

También debemos comprobar si una actualización reemplaza todo el payload o fusiona campos. Actualizar solo la metadata puede conservar el vector. Cambiar el documento puede recalcularse el embedding si la colección

tiene una función asociada, o puede dejar el vector anterior si la inferencia está fuera del motor. La semántica debe ser explícita para no terminar con un texto y una representación pertenecientes a versiones distintas.

5.2. Batching y backpressure

Enviar un vector por petición multiplica el coste de red, serialización y confirmación. Agrupar registros amortiza ese coste. Sin embargo, un lote demasiado grande puede superar límites, agotar memoria en el cliente o hacer más cara la repetición después de un fallo.

El tamaño adecuado depende de dimensión, metadata, protocolo, compresión y límites del proveedor. También depende de la concurrencia. Diez procesos enviando lotes grandes pueden saturar el servicio aunque cada proceso, por separado, parezca eficiente.

Un pipeline robusto necesita backpressure: si el motor empieza a devolver límites de tasa o aumenta su cola interna, la ingesta debe reducir ritmo, reintentar con espera exponencial y conservar qué lotes han sido confirmados. Reintentar sin IDs idempotentes puede crear duplicados; reintentar sin límite puede agravar la saturación.

5.3. Write-ahead log

Un WAL registra la intención de cambio antes de incorporarla a las estructuras definitivas. Tras una caída, el motor puede reproducir el log y reconstruir el estado que todavía no había sido compactado.

El WAL mejora durabilidad, pero no sustituye al backup. Si el operador elimina una colección y esa eliminación se registra correctamente, el WAL no tiene por qué protegernos contra la propia operación. Tampoco conserva indefinidamente versiones históricas. Su finalidad principal es recuperar el estado reciente y mantener un orden de operaciones.

En sistemas distribuidos, el log participa además en la propagación y el ordenamiento. Puede residir en disco local, un broker o almacenamiento de objetos. Cada elección cambia latencia, throughput y complejidad operativa.

5.4. Segmentos

Muchos motores agrupan registros en segmentos. Un segmento mutable recibe escrituras recientes. Cuando alcanza cierto tamaño, puede sellarse, compactarse y adquirir un índice optimizado para lectura.

Este diseño evita modificar constantemente una estructura grande. La consulta debe combinar resultados de segmentos recientes y segmentos históricos. Por eso el motor puede utilizar búsqueda exacta sobre una zona pequeña no indexada y ANN sobre los segmentos consolidados.

El número y tamaño de segmentos también afectan al rendimiento. Demasiados segmentos pequeños obligan a ejecutar y fusionar más búsquedas. Segmentos enormes tardan más en reconstruirse. Los optimizadores de fondo intentan encontrar un equilibrio mediante fusiones y umbrales.

5.5. Borrados lógicos y compactación

Eliminar inmediatamente un nodo de un grafo o reescribir un segmento puede ser caro. Una estrategia habitual consiste en marcar el registro como borrado, excluirlo de las consultas y recuperar el espacio durante una compactación posterior.

Por eso el tamaño de un contenedor o volumen puede no disminuir al borrar una colección o muchos puntos. El sistema de archivos y la base pueden conservar páginas ya asignadas para reutilizarlas. El dato deja de ser visible, pero el espacio físico no se devuelve inmediatamente al host.

Esta distinción importa en capacidad y cumplimiento. Un borrado lógico puede satisfacer la semántica de consulta y no ser suficiente para una política que exige destrucción física en un plazo concreto. La organización debe conocer cómo se propagan los borrados a réplicas, snapshots, backups y logs.

5.6. Reconstrucción y migraciones

Algunas propiedades quedan fijadas al crear la colección: dimensión, métrica, tipo de índice o ciertos parámetros de construcción. Cambiarlas puede exigir crear un recurso nuevo, reingerir y cambiar el tráfico.

Una migración segura suele utilizar versionado:

1. Se crea una colección nueva con el esquema deseado.
2. Se carga un snapshot coherente del corpus.
3. Se replican los cambios que ocurren durante la carga.
4. Se valida recuento, filtros y calidad.
5. Se cambia un alias o la configuración de la aplicación.
6. Se conserva la versión anterior durante una ventana de reversión.

Borrar y recrear la colección en el mismo nombre puede resultar aceptable en desarrollo, pero no constituye una estrategia productiva. Destruye la posibilidad de comparar y deja un periodo de indisponibilidad.

6. Persistencia, consistencia y distribución

Los términos persistencia, durabilidad, disponibilidad y consistencia describen propiedades diferentes. Mezclarlos lleva a conclusiones peligrosas, como asumir que un volumen Docker equivale a un backup o que una réplica garantiza lecturas actuales.

6.1. Persistencia y durabilidad

Persistencia significa que el estado sobrevive al proceso que lo creó. Un cliente embebido en memoria no persiste. Un archivo local o un volumen Docker sí pueden conservar datos después de reiniciar el contenedor.

Durabilidad describe la probabilidad o garantía de que una escritura confirmada sobreviva a fallos. Depende de cuándo se sincroniza el log, cuántas copias existen y qué tipo de almacenamiento se utiliza. Un archivo persistente en un único portátil puede desaparecer con el disco. Un objeto replicado por el proveedor puede ofrecer una durabilidad mucho mayor.

La confirmación de una API no debe interpretarse sin conocer el nivel al que se ha hecho duradera. Algunos sistemas permiten configurar si esperan a que la operación se aplique, se registre o se replique.

6.2. Snapshot, backup y restauración

Un snapshot captura el estado de una colección o nodo en un momento. Puede incluir vectores, payloads, configuración e índices. Un backup añade normalmente una política operativa: programación, retención, cifrado, almacenamiento separado, catálogo y procedimiento de restauración.

La copia solo es útil si puede restaurarse. Deben probarse versiones compatibles, tiempos de recuperación y dependencias externas. En una arquitectura compuesta, copiar únicamente el volumen de datos puede omitir metadatos conservados en otra base o servicio.

También conviene distinguir RPO y RTO. El **Recovery Point Objective** determina cuánto dato reciente estamos dispuestos a perder. El **Recovery Time Objective** determina cuánto tiempo puede permanecer indisponible el servicio. Hacer un snapshot semanal puede ofrecer un proceso de restauración excelente y un RPO inaceptable para una carga con escrituras continuas.

6.3. Consistencia de lectura

La consistencia indica qué versión del estado puede observar una lectura. En un sistema fuertemente consistente, una lectura posterior a una escritura confirmada observa el cambio conforme al orden definido por el sistema. En consistencia eventual, las réplicas o índices convergen si dejan de producirse cambios, pero una lectura inmediata puede devolver una versión anterior.

Entre ambos extremos existen garantías intermedias:

- **Read-your-writes** permite que un cliente observe sus propias escrituras.
- **Session consistency** mantiene esa propiedad dentro de una sesión.
- **Bounded staleness** limita cuánto puede retrasarse la vista leída.
- **Monotonic reads** evita que un cliente vuelva a una versión anterior después de haber visto una nueva.

La palabra “strong” también necesita contexto. Puede aplicarse a obtener un objeto por ID y no al conjunto de IDs elegido por una búsqueda ANN. Weaviate, por ejemplo, documenta que el nivel de consistencia de lectura afecta a la recuperación de objetos identificados, pero no implica fusionar la búsqueda contra todas las réplicas [15].

6.4. Sharding

Un shard contiene una fracción del conjunto. Permite superar la capacidad de una sola máquina y distribuir trabajo. La clave de partición determina dónde vive cada registro.

En una búsqueda global, todos los shards relevantes producen candidatos y un coordinador fusiona rankings. Si queremos un top-k global de diez elementos, solicitar exactamente diez a cada shard puede bastar para una métrica simple, pero consultas híbridas, filtros, rescoring o fusiones más complejas pueden requerir un plan distinto.

Más shards no garantizan menor latencia. Aumentan paralelismo, pero también coordinación, conexiones, metadatos y coste de fusión. Un shard muy pequeño desperdicia recursos; uno demasiado grande limita escalado y tarda más en recuperarse.

6.5. Replicación

Una réplica es otra copia de un shard. Mejora tolerancia a fallos y puede aumentar throughput de lectura. También multiplica almacenamiento y trabajo de escritura.

Si el factor de replicación es uno, perder el nodo puede volver inaccesible esa parte aunque exista un backup. El backup permite recuperar; la réplica permite continuar atendiendo. Son mecanismos complementarios.

En sistemas con consistencia configurable suelen aparecer los parámetros R, W y N: réplicas consultadas, confirmaciones de escritura y número total de copias. Como regla simplificada:

$$R + W > N$$

permite que el conjunto de lectura y el de escritura se solapen. Esta desigualdad no demuestra por sí sola una garantía completa: siguen importando resolución de conflictos, relojes, fallos parciales y qué parte de la consulta se ejecuta sobre cada réplica.

6.6. Particiones frente a shards

Una partición suele tener significado lógico para la aplicación, como país, fecha o tenant. Un shard es una unidad física de distribución administrada por el motor. Algunos productos relacionan ambos conceptos; otros los mantienen separados.

Una buena partición permite dirigir la consulta a menos datos. Una mala clave genera hotspots: una partición recibe casi todas las escrituras mientras otras permanecen ociosas. La cardinalidad, el crecimiento y los patrones de consulta deben analizarse juntos.

6.7. CAP y PACELC sin simplificaciones

El teorema CAP explica que, durante una partición de red, un sistema distribuido no puede garantizar simultáneamente disponibilidad completa y consistencia fuerte. PACELC añade que, incluso sin partición, existe un intercambio entre latencia y consistencia.

Esto no clasifica para siempre a un producto como “CP” o “AP” en todas sus operaciones. Un motor puede utilizar consenso fuerte para el esquema y consistencia eventual para los objetos. Puede permitir elegir ONE, QUORUM o ALL. Puede priorizar disponibilidad en búsquedas y exigir consenso al modificar la topología.

La pregunta útil es concreta: ante qué fallo, para qué operación y con qué configuración se ofrece cada garantía.

7. Despliegue, seguridad y observabilidad

Elegir un motor implica elegir una frontera operativa. La misma API puede ocultar arquitecturas muy diferentes según se ejecute dentro del proceso, en un contenedor único, en un clúster o como servicio gestionado.

7.1. Modo embebido

Una base embebida se ejecuta dentro de la aplicación y persiste normalmente en memoria o en un archivo. Reduce la latencia de red, elimina un servicio adicional y resulta excelente para notebooks, pruebas, aplicaciones de escritorio o edge.

La sencillez tiene límites. El ciclo de vida queda ligado al proceso. La concurrencia puede estar restringida. Escalar horizontalmente o ofrecer alta disponibilidad exige cambiar de modo. Una API compatible entre modo embebido y servidor facilita la transición, pero no garantiza que ambos compartan rendimiento, consistencia o todas las funciones.

7.2. Single node y Docker

Un servidor en un contenedor separa cliente y base. Permite observar puertos, red, persistencia y reinicios sin desplegar un clúster completo. Docker empaqueta el proceso y sus dependencias; el volumen conserva el estado fuera de la capa efímera del contenedor.

El contenedor no es una máquina virtual completa ni un backup. Si se elimina el contenedor y se mantiene el volumen, los datos pueden sobrevivir. Si se ejecuta `down --volumes`, se elimina también ese estado. La imagen descargada permanece hasta que se borra explícitamente y puede ocupar la mayor parte del espacio mostrado por Docker Desktop.

Un single node puede ser productivo para cargas moderadas si el hardware, los backups y el SLA lo permiten. No ofrece tolerancia automática a la pérdida de la máquina. Debe monitorizarse, actualizarse y restaurarse como cualquier base.

7.3. Despliegue distribuido

Un clúster permite repartir almacenamiento, consultas y réplicas. También introduce descubrimiento de nodos, consenso, balanceo, reubicación de shards, actualizaciones progresivas y diagnóstico de fallos parciales.

Kubernetes puede automatizar el ciclo de vida de procesos, pero no comprende por sí solo la semántica de la base. Un operador específico debe coordinar cambios de topología, almacenamiento y backups. Añadir nodos sin redistribuir datos no resuelve necesariamente un cuello de botella.

La arquitectura distribuida merece la pena cuando existe un requisito de capacidad, disponibilidad o throughput que no cabe en una máquina. Adoptarla por anticipación puede convertir una aplicación sencilla en una plataforma que necesita guardias y conocimiento especializado.

7.4. Servicio gestionado

En un servicio managed, el proveedor opera infraestructura, parches, una parte del escalado y determinadas garantías. El cliente conserva la responsabilidad sobre datos, esquema, credenciales, región, coste y uso correcto de la API.

La principal ventaja no es que la infraestructura deje de existir, sino que otro equipo asume una parte de su operación mediante un contrato. La contrapartida es menor control sobre algoritmos internos, ventanas de cambio y límites. También aparece dependencia de precios, regiones y funcionalidades propietarias.

Managed y self-hosted no representan calidad alta y baja. Representan distribuciones distintas de control y responsabilidad. Un equipo pequeño puede obtener más fiabilidad delegando. Una organización con plataforma propia puede necesitar aislamiento, hardware específico o control de versión.

7.5. Coste

En self-hosted se pagan máquinas, discos, red, backups y tiempo operativo. En cloud pueden facturarse almacenamiento, unidades de lectura y escritura, nodos dedicados, réplicas, transferencias e inferencia.

Una estimación mínima debe considerar:

$$C_{\text{total}} = C_{\text{almacenamiento}} + C_{\text{lecturas}} + C_{\text{escrituras}} + C_{\text{réplicas}} + C_{\text{operación}}$$

El coste no depende solo del número de vectores. La dimensión modifica el tamaño. La metadata afecta a almacenamiento y escritura. Los filtros pueden ampliar el conjunto escaneado. Las réplicas multiplican capacidad. Los backups y la transferencia también cuentan.

Un índice más grande puede encarecer una consulta serverless aunque no utilicemos nuestra propia RAM. Pinecone, por ejemplo, mide las consultas on-demand según el tamaño del namespace objetivo y factura también unidades de escritura y almacenamiento [5]. La abstracción cloud transforma el problema de memoria en una variable económica y de latencia, pero no lo elimina.

7.6. Seguridad

Un despliegue local sin autenticación es razonable en un portátil aislado. Exponerlo a una red transforma esa comodidad en una vulnerabilidad.

Un entorno productivo necesita, como mínimo:

- Autenticación de clientes y rotación de secretos.
- Autorización con mínimo privilegio para separar lectura, escritura y administración.
- TLS para proteger datos y credenciales en tránsito.

- Cifrado en reposo y gestión de claves conforme al riesgo.
- Segmentación de red y, cuando sea necesario, endpoints privados.
- Auditoría de cambios administrativos y acceso a datos.
- Políticas de retención y borrado que incluyan backups y snapshots.

Los embeddings no son automáticamente anónimos. Pueden conservar señales del contenido y participar en ataques de inferencia. El payload puede contener información personal directa. Deben protegerse con la misma seriedad que el documento de origen.

7.7. Observabilidad

La observabilidad debe cubrir infraestructura y recuperación.

En el plano técnico interesan disponibilidad, tasa de errores, latencias p50, p95 y p99, CPU, RAM, disco, IOPS, red, cola de ingesta, compactaciones, shards y réplicas. En el plano de recuperación interesan `recall@k` frente a un oráculo, relevancia de negocio, cumplimiento de filtros, distribución de scores y deriva del modelo.

Una base puede tener una latencia excelente y devolver vecinos incorrectos porque el modelo cambió. También puede reproducir perfectamente un espacio semántico deficiente. Las métricas de servicio y las de calidad deben compartir IDs, versiones y ventanas temporales para poder atribuir el problema.

8. Pinecone

Pinecone es un servicio gestionado diseñado específicamente alrededor de la recuperación vectorial. Su propuesta central consiste en ofrecer una API de base de datos sin exigir que el usuario opere nodos, discos o procesos internos.

8.1. Arquitectura serverless

La arquitectura documentada separa un plano de control global de planos de datos regionales [4]. El API gateway autentica la petición y la dirige al plano correspondiente. El plano de control mantiene proyectos, índices, usuarios y facturación. El plano de datos atiende lecturas y escrituras dentro de la región del índice.

En serverless, los registros se conservan en almacenamiento de objetos distribuido. Pinecone organiza cada namespace en archivos inmutables denominados `*slabs*`. Las rutas de lectura y escritura escalan de manera independiente. Esta separación permite que el servicio adapte capacidad a la demanda sin que el usuario configure nodos.

La consecuencia es importante: Pinecone no debe imaginarse como un HNSW residente permanentemente en la RAM de una máquina asignada al cliente. El servicio materializa su propia arquitectura de almacenamiento, indexación y caché. El algoritmo exacto y sus parámetros internos no forman parte del contrato público de la misma forma que en Qdrant o Milvus.

8.2. Índices y namespaces

El índice fija propiedades como dimensión, tipo de vector, métrica, cloud y región. Un namespace crea una frontera lógica dentro del índice y todas las operaciones de datos se dirigen a uno.

Pinecone recomienda namespaces para separar tenants porque se almacenan de manera independiente y reducen el conjunto escaneado [6]. Esta elección también afecta al coste: en on-demand, la consulta se factura según el tamaño del namespace objetivo. Utilizar un único namespace enorme y filtrar por usuario puede ser más caro y menos seguro que separar tenants.

El namespace no sustituye todos los modelos de autorización. La aplicación debe asegurarse de seleccionar el correcto y utilizar credenciales con permisos adecuados. Sin embargo, ofrece una frontera más fuerte que confiar exclusivamente en un campo de metadata.

8.3. Datos y filtros

El modelo vectorial clásico almacena ID, vector y metadata. Pinecone admite operadores de comparación y composición sobre esos metadatos. Las APIs más recientes incorporan además un modelo de documentos con campos densos, sparse, texto completo y campos filtrables [7].

La capacidad no obliga a utilizar inferencia integrada. En un flujo BYOV, la aplicación calcula embeddings y conserva control sobre modelo y preprocesamiento. La inferencia alojada simplifica arquitectura, pero añade otro servicio facturable y acopla la representación al catálogo disponible del proveedor.

Pinecone no expone al usuario una selección de HNSW, IVF o DiskANN para serverless. Se eligen dimensión y métrica, y se evalúa el resultado. Esta opacidad reduce tuning y mantenimiento. También impide aplicar directamente una configuración aprendida sobre un índice concreto o explicar un comportamiento mediante parámetros internos.

8.4. Consistencia y frescura

Pinecone documenta consistencia eventual: puede existir un retraso breve entre una escritura y su visibilidad en consultas [9]. El servicio ofrece recuentos y números de secuencia del log para comprobar frescura en serverless.

Esto obliga a diseñar read-after-write de forma consciente. Una interfaz que crea un registro y lo busca inmediatamente puede necesitar reintentos. Dormir una cantidad fija es frágil; comprobar una condición con deadline produce un comportamiento más observable.

La consistencia eventual no implica pérdida. Describe cuándo una lectura puede observar el cambio. Durabilidad, backup y visibilidad son propiedades distintas.

8.5. Capacidad y facturación

Los índices on-demand facturan almacenamiento, Read Units y Write Units [5]. El coste de una consulta crece con el tamaño del namespace objetivo. Las escrituras dependen del tamaño de la petición y del registro existente cuando se modifica. En cargas sostenidas de lectura, Pinecone también ofrece Dedicated Read Nodes, con hardware aprovisionado para lecturas previsibles [10].

Este modelo favorece cargas variables: no se paga un nodo dedicado ocioso en on-demand. En una carga constante y alta, las unidades consumidas pueden ser menos previsibles que una capacidad reservada. La estimación debe utilizar el tamaño real del namespace y el patrón de operaciones, no solo el número de registros.

La dimensión afecta dos veces. Aumenta almacenamiento y tamaño de las escrituras. También incrementa el namespace sobre el que se calculan lecturas. Los metadatos anchos producen un efecto parecido.

8.6. Operación y seguridad

Pinecone gestiona infraestructura, actualizaciones y escalado del servicio. Ofrece API keys con roles, cifrado en tránsito y reposo, backups en planes compatibles, endpoints privados, RBAC y opciones de claves gestionadas por el cliente [11].

El usuario sigue siendo responsable de rotar claves, elegir región, modelar namespaces, controlar límites y evitar borrados accidentales. La protección frente a eliminación debe habilitarse donde sea apropiada. Managed reduce la superficie operativa; no elimina gobierno ni observabilidad.

Pinecone Local permite ejecutar un emulador para desarrollo [29]. Sirve para validar integración sin consumir cloud, pero no reproduce la arquitectura ni el rendimiento del servicio. Un emulador no debe utilizarse para estimar latencia, capacidad o disponibilidad productivas.

8.7. Fortalezas y costes de la decisión

La principal fortaleza de Pinecone es la reducción de operación. El equipo consume una API, no administra un clúster ni elige un índice ANN. Serverless se adapta bien a demanda irregular y los namespaces ofrecen una unidad clara de aislamiento.

La contrapartida es el control limitado sobre el algoritmo, la infraestructura y determinadas decisiones de rendimiento. Existe dependencia del proveedor, sus regiones, cuotas, modelo de precios y evolución de API. El diagnóstico se apoya en métricas y contratos expuestos, no en inspeccionar segmentos o reconstruir el índice con otro parámetro.

Pinecone resulta especialmente atractivo cuando la prioridad es poner en producción una recuperación vectorial sin crear una capacidad interna de operación. Resulta menos natural cuando se exige desplegar en una infraestructura aislada, controlar exactamente el algoritmo o mantener portabilidad completa del plano de control.

9. Chroma

Chroma nació con una experiencia centrada en desarrolladores: crear una colección, añadir documentos y consultarla con muy poca configuración. Esa sencillez no significa que carezca de arquitectura; significa que intenta mantenerla fuera del camino hasta que el despliegue la necesita.

9.1. Modos de despliegue

Chroma documenta tres modos principales: local embebido, single node y distribuido [8]. El cliente efímero conserva estado en memoria. `PersistentClient` utiliza una ruta local. `HttpClient` separa cliente y servidor. Chroma Cloud ofrece la arquitectura distribuida como servicio gestionado.

El modo embebido resulta cómodo para notebooks, tests y aplicaciones pequeñas. El modo cliente-servidor permite varios procesos y persistencia independiente. El distribuido separa servicios y almacenamiento para escalar. La API común reduce fricción al avanzar entre modos, pero las propiedades físicas cambian.

En particular, no debe extrapolarse el rendimiento de un cliente local a Chroma Cloud. El índice, la caché, la latencia de red y la consistencia pueden ser diferentes aunque el método de consulta se parezca.

9.2. Modelo de datos

Chroma organiza el sistema en tenants, databases y collections. Cada registro posee ID, embedding, documento y metadata opcional. Esta representación resulta natural para aplicaciones documentales porque el texto no tiene que esconderse dentro de un JSON genérico.

La colección puede asociarse a una función de embeddings. Si se entregan documentos sin vectores, Chroma puede calcularlos. Si la aplicación aporta ambos, los almacena sin reembedding [30]. Esta flexibilidad permite trabajar con inferencia integrada o BYOV.

Existe una diferencia semántica entre `add`, `update` y `upsert`. `add` no reemplaza un ID existente. `update` modifica registros conocidos. `upsert` crea o actualiza [31]. Elegir el método adecuado permite detectar errores de identidad.

9.3. Índices

Chroma Single Node utiliza HNSW [32]. Permite configurar la distancia, `max_neighbors`, `ef_construction`, `ef_search`, número de hilos, batch interno y umbral de sincronización. Algunos parámetros quedan fijados al crear la colección y otros pueden modificarse.

Chroma Distributed y Chroma Cloud utilizan SPANN. Este índice divide el conjunto en clusters amplios y utiliza estructuras locales para reducir memoria y accesos en escenarios grandes. La documentación actual indica que la configuración detallada de SPANN no se expone al usuario [32].

Esta diferencia impide decir simplemente “Chroma usa HNSW” sin especificar modo. También recuerda que la portabilidad de API no equivale a identidad del motor físico.

9.4. Arquitectura distribuida

La arquitectura distribuida documenta cinco componentes: gateway, log, query executor, compactor y system database [33].

El gateway autentica, limita y planifica peticiones. El log registra escrituras antes de confirmarlas. El query executor atiende búsqueda vectorial, texto y metadatos. El compactor materializa nuevas versiones de índices. La system database conserva tenants, bases, colecciones y metadatos del clúster.

Los logs e índices residen en almacenamiento de objetos; una base SQL conserva el catálogo; SSD locales actúan como caché. Este diseño separa cómputo y almacenamiento, pero introduce posibles **cold starts**: una colección que no está en caché debe recuperar datos desde object storage. Cuando la caché se calienta, las lecturas evitan repetir ese coste.

9.5. Filtros

Chroma utiliza diccionarios `where` para metadatos y `where_document` para contenido. Admite igualdad, comparaciones, operadores lógicos y, en las versiones actuales, arrays bajo restricciones de tipo [34].

La sintaxis es fácil de construir desde Python, aunque está ligada al contrato de Chroma. Una capa genérica debe traducir correctamente tipos, operadores y nulos. El hecho de que el filtro sea un diccionario no significa que pueda trasladarse sin cambios a Qdrant o Weaviate.

9.6. Persistencia y operación

En single node, Chroma delega parte de la persistencia estructurada en SQLite y mantiene el índice vectorial en el directorio persistente. El `sync_threshold` controla cuándo se sincroniza HNSW. Un volumen Docker debe montar ese directorio para sobrevivir al contenedor.

Eliminar una colección borra su contenido lógico, pero no elimina el contenedor, la imagen ni el volumen. El archivo o volumen puede conservar espacio asignado para reutilizarlo. Para liberar todo el entorno local hay que detener el compose y eliminar explícitamente sus volúmenes; la imagen sigue siendo un artefacto separado.

En producción deben revisarse autenticación, backup, concurrencia y topología del modo concreto. La sencillez del cliente no debe confundirse con ausencia de operación.

9.7. Fortalezas y costes de la decisión

Chroma destaca por una barrera de entrada baja, una representación directa de documentos y la posibilidad de empezar dentro del proceso. Resulta cómodo para prototipos, enseñanza y aplicaciones que valoran una API pequeña.

El intercambio aparece al crecer. El modo single node no ofrece las mismas garantías que la arquitectura distribuida, y pasar a Cloud cambia el índice de HNSW a SPANN. Equipos que necesitan un control muy amplio de índices, consistencia o topología pueden encontrar más superficie en Milvus, Weaviate o Qdrant.

Chroma es una buena elección cuando la productividad y el modelo documental pesan más que disponer de muchas familias ANN. Debe evaluarse con especial cuidado cuando el requisito depende de una función avanzada o de una garantía operativa específica que no esté igualmente disponible en todos sus modos.

10. Weaviate

Weaviate combina una base de objetos, índices vectoriales e índices invertidos. Esta combinación explica buena parte de su propuesta: la búsqueda semántica no se trata como una función aislada, sino como un acceso que convive con propiedades tipadas, filtros y búsqueda léxica.

10.1. Objetos y colecciones

Una colección define propiedades, tipos, vectorizadores, índice vectorial, índices invertidos, sharding y replicación. Cada objeto posee UUID, propiedades y uno o varios vectores.

Weaviate puede ejecutar módulos de vectorización o aceptar vectores aportados por el cliente [14]. BYOV conserva el control sobre el encoder y evita que una actualización del módulo modifique silenciosamente la representación. Los módulos integrados reducen código y pueden mantener el vector sincronizado con el objeto.

El esquema tipado obliga a decidir qué es texto, número, fecha o booleano. Esa explicitud mejora filtros y validación. A cambio, modificar el esquema o la vectorización requiere una migración más consciente que almacenar payload JSON completamente libre.

10.2. Almacenamiento dentro de un shard

Cada shard contiene un object store, un inverted index y un vector index [16]. El almacenamiento de objetos y estructuras invertidas utiliza un enfoque LSM-tree: las escrituras llegan a una memtable y se materializan posteriormente en segmentos ordenados.

El LSM-tree favorece ingestión secuencial y evita reescrituras aleatorias continuas. Las lecturas deben consultar la memtable y segmentos recientes hasta encontrar la versión válida. Las compactaciones fusionan segmentos y recuperan espacio.

El índice vectorial convive físicamente con el índice invertido dentro del shard. Esta cercanía permite que los filtros produzcan una lista de candidatos que la búsqueda HNSW puede utilizar sin ejecutar necesariamente un postfiltro.

10.3. Índices vectoriales

La familia actual documenta HNSW, flat, dynamic y HFresh [13].

HNSW es la opción general para colecciones grandes y top-k reducido. Weaviate expone `ef`, `efConstruction`, `maxConnections`, `distancia`, `caché`, cuantización y limpieza de tombstones. Algunos parámetros son mutables y otros requieren reconstrucción.

Flat compara contra los vectores del conjunto. Puede ser adecuado para colecciones pequeñas o tenants con pocos objetos, donde el grafo consumiría más de lo que ahorra.

Dynamic comienza con un índice plano y cambia a HNSW al superar un umbral. Intenta resolver el problema de colecciones pequeñas que crecen, aunque debe considerarse el estado de madurez documentado para la versión utilizada.

HFresh es una opción orientada a arquitecturas distribuidas y eficiencia de memoria. Utiliza una organización por clusters y HNSW para sus centroides. No debe asumirse que comparte parámetros y comportamiento con el HNSW clásico.

10.4. Índice invertido y filtros

Weaviate puede crear índices distintos por propiedad para búsqueda y filtrado [17]. `indexSearchable` sirve a búsqueda textual, mientras `indexFilterable` optimiza coincidencias y utiliza Roaring Bitmaps.

En una búsqueda vectorial filtrada, el índice invertido genera un allow-list de IDs. HNSW explora el grafo y solo incorpora al resultado IDs permitidos. La estrategia `sweeping` recorre el grafo respetando esa lista. ACORN mejora casos en los que el filtro es restrictivo o está poco correlacionado con la geometría, utilizando saltos multihop y puntos de entrada adicionales [12].

Esta integración constituye una fortaleza cuando los filtros forman parte central del patrón de consulta. También consume almacenamiento: habilitar varios índices sobre muchas propiedades incrementa el coste de ingesta y persistencia. El esquema debe reflejar las operaciones reales.

10.5. HTTP, GraphQL y gRPC

Weaviate expone interfaces HTTP/REST, GraphQL y gRPC. El cliente Python v4 utiliza gRPC para numerosas operaciones y necesita que el puerto 50051 sea accesible [18].

gRPC utiliza normalmente HTTP/2 y Protocol Buffers. Los mensajes son binarios y el contrato está tipado. HTTP con JSON resulta más fácil de inspeccionar manualmente; gRPC reduce overhead y facilita streaming y multiplexación.

Esto es una decisión de comunicación, no del algoritmo ANN. Una búsqueda enviada por gRPC puede ejecutar exactamente el mismo HNSW que una petición equivalente. El protocolo afecta serialización, conexiones y depuración, no la geometría.

10.6. Sharding, replicación y consistencia

El sharding distribuye objetos; la replicación conserva copias. Weaviate utiliza Raft para metadatos del clúster, como definiciones de colecciones, y un diseño leaderless para objetos [15].

Las operaciones de datos pueden utilizar niveles `ONE`, `QUORUM` o `ALL`. Un nivel bajo favorece disponibilidad y latencia; `ALL` espera más confirmaciones. Las reparaciones asíncronas y read-repair ayudan a converger réplicas.

La precisión importante es que el nivel de lectura no obliga a ejecutar la búsqueda ANN contra todas las réplicas y fusionarlas. Se aplica a la obtención de los objetos identificados. Esta distinción evita atribuir a `ALL` una garantía que no describe.

10.7. Fortalezas y costes de la decisión

Weaviate ofrece un modelo rico, búsqueda vectorial, filtros y capacidades léxicas dentro del mismo motor. La integración entre HNSW e índice invertido es especialmente atractiva para recuperación condicionada por propiedades. El esquema tipado y los módulos reducen trabajo de aplicación.

La amplitud añade complejidad. Hay más decisiones sobre vectorizadores, propiedades, índices invertidos, tipo de vector index, sharding y replicación. Un clúster exige comprender dos modelos de consistencia: metadatos y objetos. La memoria de HNSW y de los índices escalares debe dimensionarse.

Weaviate encaja cuando la entidad con propiedades y la búsqueda filtrada son parte central del sistema. Puede resultar excesivo para una colección pequeña que solo necesita similitud y persistencia sencilla.

11. Milvus

Milvus es el motor que expone con mayor claridad una arquitectura orientada a gran escala y una amplia elección de índices. Su diseño separa almacenamiento y cómputo y divide el trabajo entre componentes especializados.

11.1. Tres modos de despliegue

Milvus Lite se ejecuta como biblioteca Python y persiste en un archivo. Comparte gran parte de la API, pero está pensado para escalas pequeñas, notebooks y edge [19].

Milvus Standalone empaqueta el servidor en una máquina. En el compose habitual aparecen además etcd para metadatos y MinIO para objetos. Es una opción intermedia cuando se quiere un servicio real sin operar Kubernetes.

Milvus Distributed separa componentes sobre un clúster y permite escalar lecturas, escrituras y almacenamiento de forma independiente [35]. Es la opción con mayor capacidad y también con mayor carga operativa.

La compatibilidad de API facilita el desarrollo, pero no hace equivalentes los modos. Lite no reproduce red, coordinación ni todas las garantías distribuidas. Standalone no proporciona alta disponibilidad por el simple hecho de utilizar varios contenedores auxiliares.

11.2. Arquitectura por capas

La arquitectura distribuida se organiza en acceso, coordinación, workers y almacenamiento [21].

Los proxies de acceso son stateless. Validan peticiones, enrutan y fusionan resultados. El coordinador administra topología, DDL, timestamps y planificación. Los Streaming Nodes procesan escrituras recientes y mantienen datos crecientes. Los Query Nodes atienden datos históricos cargados desde object storage. Los Data Nodes ejecutan compactación y construcción de índices.

El almacenamiento separa metadata, WAL y objetos. etcd conserva el catálogo y la coordinación. El object storage guarda binlogs, índices y archivos de datos. El WAL establece el orden y permite recuperación.

Esta separación favorece elasticidad: aumentar Query Nodes puede atender una carga de lectura sin multiplicar necesariamente almacenamiento. A cambio, aparecen más componentes, redes y estados que monitorizar.

11.3. Growing y sealed segments

Los datos recién escritos forman **growing segments**. Deben ser consultables antes de que exista el índice definitivo. Cuando alcanzan un umbral, se sellan. Los Data Nodes compactan y construyen índices sobre segmentos sellados; los Query Nodes cargan esas estructuras.

La búsqueda combina datos recientes e históricos. Este modelo explica por qué aceptar una inserción, persistirla y verla a través del índice no son el mismo instante. Los índices intermedios pueden acelerar segmentos todavía crecientes, pero el camino sigue siendo diferente al de datos consolidados.

11.4. Esquema e índices escalares

Milvus permite definir clave primaria, campos numéricos, cadenas, JSON, arrays y uno o varios campos vectoriales. El esquema explícito aporta control sobre tipos y límites.

Los filtros se expresan mediante una sintaxis propia. Los campos escalares pueden disponer de índices invertidos, bitmap, trie u otras estructuras según el tipo [20]. Al igual que en los demás motores, indexar un campo acelera lectura a costa de almacenamiento y escritura.

Las particiones permiten agrupar entidades y evitar consultar toda la colección cuando la clave es conocida. Las `partition keys` automatizan parte de ese enrutamiento. Una mala partición puede concentrar carga o crear demasiadas unidades pequeñas.

11.5. Oferta de índices vectoriales

Milvus 2.6 documenta una gama especialmente amplia: FLAT, IVF_FLAT, variantes IVF cuantizadas, HNSW, HNSW cuantizado, DiskANN, SCANN y opciones GPU, entre otras [20].

Esta oferta permite adaptar el índice a top-k, memoria, disco, aceleradores y recall. HNSW funciona bien cuando los datos y el grafo caben en memoria. IVF permite controlar listas y probes. DiskANN utiliza SSD para ampliar capacidad. Las variantes cuantizadas intercambian precisión por memoria. GPU_CAGRA y familias GPU aprovechan hardware específico.

La fortaleza implica una obligación: el motor no puede elegir por nosotros el objetivo de negocio. Necesitamos construir un baseline exacto, probar índices con filtros representativos y medir construcción, búsqueda, memoria y calidad. Una matriz genérica de recomendaciones orienta, pero no sustituye la evaluación.

11.6. Consistencia

Milvus ofrece Strong, Bounded Staleness, Session y Eventually [22]. El nivel se traduce en un `GuaranteeTs` que indica hasta qué timestamp deben haber avanzado los Query Nodes antes de atender la consulta.

Strong espera una vista más reciente y puede añadir latencia. Bounded permite un retraso controlado. Session garantiza que el cliente vea sus propias escrituras dentro de la sesión. Eventually prioriza respuesta sobre actualidad inmediata.

Esta variedad resulta valiosa cuando diferentes cargas toleran frescura distinta. También obliga a elegir. Utilizar el valor por defecto sin conocerlo convierte una decisión de consistencia en un accidente.

11.7. Recursos y operación

La separación de cómputo y object storage permite escalar, pero la ruta de consulta necesita cargar segmentos e índices. Cachés, memoria de Query Nodes y latencia del almacenamiento influyen de manera decisiva.

En Standalone, etcd, MinIO y Milvus forman una unidad operativa. Los tres volúmenes importan. Copiar solo `/var/lib/milvus` puede no ser un backup coherente si se omite metadata u objetos.

En Distributed, Kubernetes, balanceo, upgrades y observabilidad forman parte del coste. Zilliz Cloud ofrece la ruta gestionada para delegar esa arquitectura.

11.8. Fortalezas y costes de la decisión

Milvus destaca por escala, separación de cómputo y almacenamiento y elección de índices. Resulta adecuado para equipos que necesitan controlar arquitectura, hardware y algoritmos, o que prevén cargas muy grandes y heterogéneas.

La contrapartida es complejidad. Incluso Standalone utiliza varios servicios de persistencia. Distributed exige una plataforma madura. La variedad de índices puede producir configuraciones excelentes o decisiones difíciles de mantener.

Milvus es una elección sólida cuando la flexibilidad algorítmica y la escalabilidad justifican la inversión operativa. Para un catálogo pequeño con una única búsqueda HNSW, puede ofrecer mucha más superficie de la necesaria.

12. Qdrant

Qdrant es una base vectorial escrita en Rust que organiza cada colección como un conjunto de puntos. Cada punto contiene un ID, uno o varios vectores y un payload JSON. Su diseño intenta mantener cerca la búsqueda vectorial y el filtrado estructurado.

12.1. Colecciones, puntos y vectores nombrados

Una colección fija dimensiones y métricas. Los vectores nombrados permiten que un mismo punto tenga varias representaciones, cada una con configuración propia [23].

El payload acepta datos estructurados y rutas anidadas. Los filtros se construyen mediante condiciones `must`, `should` y `must_not`. Esta forma es flexible para modelos documentales y permite mantener la entidad completa junto al vector.

La flexibilidad no elimina el esquema físico. Los campos usados en filtros frecuentes deben tener payload indexes con un tipo coherente. Un JSON mal tipado puede funcionar en desarrollo y convertirse en un scan costoso al crecer.

12.2. WAL y segmentos

Qdrant registra primero las operaciones en un WAL y después las aplica al almacenamiento [24]. La colección se divide en segmentos que contienen vectores, payloads e índices.

Los segmentos pequeños pueden permanecer sin HNSW y utilizar búsqueda exacta. Cuando superan umbrales, el optimizador construye índices o los mueve a mmap. Otros optimizadores fusionan segmentos y recuperan espacio ocupado por borrados [25].

Durante una reconstrucción, un proxy copy-on-write mantiene el segmento disponible y dirige cambios recientes a una zona prioritaria. Esta arquitectura permite optimizar sin detener completamente lecturas y escrituras, aunque consume recursos temporales.

12.3. HNSW filtrable

Qdrant utiliza HNSW como índice denso [26]. Expone `m`, `ef_construct`, `full_scan_threshold` y `ef` de consulta. También permite decidir si vectores e índice residen en disco.

Su característica distintiva es la integración con payload indexes. Al construir el grafo después de crear esos índices, Qdrant puede añadir conexiones relacionadas con valores del payload. Esto evita que un filtro intermedio rompa la navegabilidad del grafo.

Las versiones actuales incorporan además una estrategia ACORN para combinaciones de filtros especialmente restrictivas. Como siempre, estas capacidades necesitan evaluación: un índice de payload que no corresponde a las consultas solo consume recursos.

El orden de creación importa. Si el payload index aparece después de construir HNSW, el grafo existente no adquiere automáticamente todas las conexiones conscientes del filtro hasta una reconstrucción.

12.4. Memoria, disco y cuantización

Qdrant permite mantener vectores o payloads en disco, utilizar mmap y conservar HNSW en memoria o también on disk. La configuración determina cuánta RAM se intercambia por I/O.

La familia 1.18 documenta cuantización scalar, binary, product y TurboQuant [27]. Scalar transforma componentes de 32 a 8 bits. Binary reduce cada dimensión a uno o pocos bits y funciona mejor con vectores altos y distribuciones adecuadas. Product Quantization busca compresión extrema. TurboQuant aplica rotación y cuantización asimétrica para mejorar el equilibrio en diferentes distribuciones.

El rescoring con vectores originales puede recuperar precisión. Si esos originales viven en disco, el refinamiento añade lecturas aleatorias. El parámetro de oversampling controla cuántos candidatos se reevalúan. La configuración óptima depende de si el cuello es RAM, CPU o I/O.

12.5. Shards, réplicas y consistencia

Qdrant permite definir número de shards, factor de replicación y factor de escritura. Los shards pueden distribuirse y replicarse entre nodos [28].

La replicación mejora disponibilidad, pero el valor por defecto de un despliegue simple puede ser uno. Para producción, la documentación recomienda más de una copia cuando se necesita tolerancia a fallos.

Las escrituras ofrecen controles de espera y ordenamiento. Los niveles de consistencia de lectura determinan cuántas réplicas participan según la operación. Estas opciones deben configurarse junto con el factor de réplica; pedir quorum en una colección sin copias no crea redundancia.

12.6. Snapshots y seguridad

Los snapshots contienen configuración, puntos y payloads de una colección en un nodo [36]. En un clúster distribuido deben coordinarse por nodo o utilizar el mecanismo de backup apropiado. La compatibilidad de versiones importa durante la restauración.

El Qdrant open source local arranca sin autenticación si no se configura. Para exponerlo se deben habilitar API keys y TLS, o situarlo detrás de una capa segura. También dispone de clave de solo lectura y control granular mediante JWT [37].

Qdrant Cloud ofrece clusters gestionados. Hybrid y Private Cloud cambian la frontera de responsabilidad: el plano de datos puede vivir en infraestructura del cliente, con diferentes grados de conexión al plano de gestión.

12.7. Fortalezas y costes de la decisión

Qdrant ofrece una API directa, payload flexible, HNSW configurable y una integración profunda entre filtros e índice. Las opciones de cuantización y almacenamiento permiten controlar RAM frente a disco sin cambiar de familia ANN.

La limitación algorítmica es precisamente esa concentración en HNSW para dense vectors. Si el requisito exige comparar IVF, DiskANN o índices GPU dentro del mismo motor, Milvus ofrece más alternativas. La flexibilidad de payload requiere diseñar índices escalares y vigilar tipos.

Qdrant encaja especialmente bien cuando los filtros estructurados son importantes y se desea un motor autogestionable con una superficie operativa menor que Milvus Distributed. Sigue necesitando backups, seguridad y replicación explícita; ejecutar un contenedor único no convierte el servicio en alta disponibilidad.

13. LangChain como capa de abstracción

LangChain no es una base de datos vectorial. Es un framework que define interfaces comunes para modelos, documentos, vector stores, retrievers y composición. Su utilidad aparece por encima del motor, cuando una aplicación necesita intercambiar componentes o construir un pipeline.

13.1. Document

`Document` representa contenido textual, metadata e ID [38]. Esta forma permite que una aplicación reciba resultados de Chroma o Qdrant sin trabajar constantemente con la respuesta nativa.

La conversión también puede perder información. El resultado del proveedor puede incluir versión, shard, score nativo, payload complejo o telemetría que el adaptador no conserva. Antes de adoptar la representación común hay que comprobar qué campos sobreviven.

13.2. Vector store y retriever

La interfaz `VectorStore` reúne almacenamiento y búsqueda. Un retriever es más general: recibe una consulta y devuelve documentos. No está obligado a almacenar ni a utilizar vectores [39].

`as_retriever()` permite tratar un vector store como recuperador e invocarlo mediante `invoke()`. Esto facilita componer un pipeline sin acoplar el consumidor a un proveedor. También limita la superficie al denominador común.

Una aplicación que solo necesita `query -> list[Document]` puede beneficiarse mucho. Una herramienta administrativa que crea shards, restaura snapshots o ajusta HNSW no debería ocultarse detrás de esa interfaz.

13.3. MMR

Maximal Marginal Relevance intenta equilibrar relevancia y diversidad. Selecciona iterativamente un documento parecido a la consulta, penalizando candidatos redundantes respecto a los ya elegidos.

Una formulación habitual es:

$$\arg \max_{d \in R \setminus S} \left[\lambda \text{sim}(d, q) - (1 - \lambda) \max_{s \in S} \text{sim}(d, s) \right]$$

R es el conjunto candidato y S los documentos ya seleccionados. MMR no mejora el índice ni descubre elementos que nunca entraron en R . `fetch_k` define el universo disponible para diversificar. Por eso debe entenderse como una transformación posterior del ranking.

13.4. La neutralidad tiene límites

Chroma y Qdrant pueden compartir métodos de similitud, pero sus filtros siguen siendo distintos. Chroma recibe un diccionario sobre metadata plana. Qdrant necesita un objeto `Filter` y una ruta de payload. Weaviate tiene propiedades tipadas. Pinecone utiliza namespaces y su propia gramática.

Scores, consistencia, batches, errores, timeouts y administración tampoco se homogeneizan por completo. Si un adaptador convierte distancia en relevancia, la aplicación debe conocer la transformación antes de fijar un umbral.

El framework añade además su propio ciclo de versiones. Puede existir una versión nueva del SDK nativo que todavía no sea compatible con la integración. La portabilidad requiere fijar versiones y ejecutar pruebas de contrato, no limitarse a cambiar una cadena de conexión.

13.5. Cuándo utilizar LangChain

LangChain aporta valor cuando queremos:

- Componer retrievers y transformaciones.
- Intercambiar rápidamente un componente durante exploración.
- Estandarizar la salida en documentos.
- Encadenar recuperación, formateo y consumidores posteriores.

El SDK nativo sigue siendo preferible para:

- Crear y administrar recursos.
- Configurar índices y consistencia.
- Diagnosticar errores y rendimiento.
- Utilizar funciones avanzadas del proveedor.
- Mantener control exacto sobre scores, filtros y respuestas.

La arquitectura más razonable suele combinar ambos niveles: administración y observabilidad mediante el SDK nativo; composición de recuperación mediante la abstracción cuando realmente reduce acoplamiento.

14. Cómo elegir una base de datos vectorial

La elección no debería empezar por una tabla de logotipos. Debe comenzar por el problema, convertirlo en requisitos medibles y comprobar qué motor satisface el conjunto completo.

14.1. Requisitos de recuperación

Necesitamos saber:

- Cuántos vectores existen hoy y cuánto crecerán.
- Qué dimensión y tipos de representación se almacenan.
- Qué top-k necesita la aplicación.
- Qué recall mínimo es aceptable.
- Qué filtros aparecen y con qué selectividad.
- Si se necesita búsqueda sparse, híbrida o multivector.
- Cuánto cambia el corpus.

Una colección de 100.000 elementos con filtros muy restrictivos puede necesitar una estrategia distinta a cien millones de vectores casi inmutables. El número total no describe por sí solo la carga.

14.2. Requisitos operativos

También debemos fijar:

- Throughput de lectura y escritura.
- Latencia p95 y p99.
- Frescura necesaria después de una mutación.
- Disponibilidad y tolerancia a fallos.
- RPO y RTO.
- Regiones, residencia y conectividad privada.
- Experiencia operativa del equipo.
- Presupuesto y previsibilidad del coste.

Estas variables pueden descartar una opción antes de evaluar su ANN. Un servicio que no puede desplegarse en la región requerida no se vuelve válido por obtener gran recall.

14.3. Comparación razonada

Motor	Decisión arquitectónica dominante	Control ANN	Frontera operativa habitual
Pinecone	Servicio serverless con almacenamiento separado y namespaces	Algoritmo interno delegado	Managed
Chroma	API documental sencilla y continuidad entre local, single node y distribuido	HNSW local; SPANN distribuido	Embebido, servidor o Cloud
Weaviate	Objetos tipados, índice vectorial e invertido dentro del shard	HNSW, flat, dynamic y HFresh	Self-hosted o Cloud
Milvus	Separación de cómputo y almacenamiento con workers especializados	Oferta amplia: FLAT, IVF, HNSW, DiskANN y más	Lite, Standalone, Distributed o Zilliz
Qdrant	Puntos y payloads con HNSW consciente de filtros	HNSW configurable y cuantización	Local, distribuido o Cloud

Esta tabla no puntúa motores. Resume dónde coloca cada uno el centro de gravedad.

Pinecone reduce operación y control interno. Chroma reduce fricción inicial. Weaviate integra objetos, filtros y búsqueda. Milvus maximiza opciones y escala arquitectónica. Qdrant concentra una superficie directa alrededor de HNSW y payload.

14.4. Prueba representativa

La decisión final necesita una prueba que mantenga constantes modelo, datos y consultas. Debe medir:

- Calidad frente a búsqueda exacta y juicios de relevancia.
- Latencia bajo concurrencia realista.
- Ingesta inicial y actualizaciones continuas.
- Filtros de selectividad alta, media y baja.
- Uso de memoria, disco y red.
- Tiempo de recuperación tras reinicio o restauración.
- Coste bajo el patrón esperado.

No debe construirse un ranking de velocidad mezclando cloud con localhost. La red, el hardware y la carga son variables del experimento. Si se desea comparar motores, deben desplegarse en condiciones equivalentes o describirse por separado.

14.5. Evitar la sobrearquitectura

Si el conjunto cabe en memoria, cambia poco y una única aplicación lo consulta, FAISS más una capa persistente sencilla puede ser suficiente. Adoptar un clúster distribuido no mejora automáticamente la relevancia.

La base vectorial se justifica cuando necesitamos que el motor asuma responsabilidades reales: metadatos y filtros, mutaciones concurrentes, persistencia, aislamiento, escalado, seguridad, backups u operación multiusuario.

Elegir el sistema más grande “por si acaso” suele trasladar el riesgo desde la capacidad hacia la complejidad. Elegir el más sencillo ignorando el crecimiento hace lo contrario. La decisión correcta es la que satisface el horizonte razonable con un coste que el equipo puede operar.

14.6. Cierre

Los embeddings construyen el espacio. La métrica define la comparación. El ANN reduce el trabajo. La base conserva, distribuye y gobierna el estado. El framework compone interfaces. La aplicación decide qué resultado tiene valor.

Separar esas capas no es un ejercicio académico. Es la única forma de atribuir errores y tomar decisiones defendibles. Una base no corrige un embedding malo. Un framework no vuelve idénticos dos motores. Una réplica no sustituye un backup. Un volumen Docker no crea alta disponibilidad. Una API sencilla no elimina las garantías que el sistema debe cumplir.

Comprender una base de datos vectorial consiste, en último término, en saber qué responsabilidad asume y cuál sigue siendo nuestra. A partir de ahí, sus características dejan de ser una lista comercial y se convierten en consecuencias razonables de una arquitectura.

15. Referencias

- [1] Johnson, Douze y Jégou. [Billion-scale similarity search with GPUs](#).
- [2] Malkov y Yashunin. [Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs](#).
- [3] Subramanya et al. [DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node](#).
- [4] Pinecone. [Database architecture](#).
- [5] Pinecone. [Understanding cost](#).
- [6] Pinecone. [Implement multitenancy](#).
- [7] Pinecone. [Concepts](#).
- [8] Chroma. [Architecture overview](#).
- [9] Pinecone. [Check data freshness](#).
- [10] Pinecone. [Dedicated Read Nodes](#).
- [11] Pinecone. [Security overview](#).
- [12] Weaviate. [Filtering](#).
- [13] Weaviate. [Vector index configuration](#).
- [14] Weaviate. [Bring your own vectors](#).
- [15] Weaviate. [Consistency](#).
- [16] Weaviate. [Storage](#).
- [17] Weaviate. [Inverted indexes](#).
- [18] Weaviate. [Python client](#).
- [19] Milvus. [Run Milvus Lite locally](#).
- [20] Milvus. [Index explained](#).
- [21] Milvus. [Architecture overview](#).
- [22] Milvus. [Consistency](#).

- [23] Qdrant. [Collections](#).
- [24] Qdrant. [Storage](#).
- [25] Qdrant. [Optimizer](#).
- [26] Qdrant. [Indexing](#).
- [27] Qdrant. [Quantization](#).
- [28] Qdrant. [Distributed deployment](#).
- [29] Pinecone. [Local development with Pinecone Local](#).
- [30] Chroma. [Adding data to collections](#).
- [31] Chroma. [Update data](#).
- [32] Chroma. [Configure collections](#).
- [33] Chroma. [Distributed architecture](#).
- [34] Chroma. [Metadata filtering](#).
- [35] Milvus. [Deployment options](#).
- [36] Qdrant. [Snapshots](#).
- [37] Qdrant. [Security](#).
- [38] LangChain. [Document](#).
- [39] LangChain. [Retrievers](#).

La matriz ampliada de versiones y documentación complementaria se conserva en `docs/REFERENCIAS.md`.